



**KINGDOM OF MOROCCO**

International University of Rabat (UIR)

School of Data Science and Digital Engineering

Data Engineering Program — 4th Year

---

# FX P&L Trading Room Application

Design, Development and Automation of a Data Engineering Platform for Foreign Exchange Profit and Loss Monitoring, Calculation, Risk Assessment, and Strategy Backtesting

---

**Accounting Currency: MAD (Moroccan Dirham)**


| | |
|---|---|
| **Prepared by:** | Mohamed Taha El Qassid |
| **Program:** | Data Engineering, 4th Year |
| **Internship Host Company:** | DXC Technology |
| **Academic Supervisor:** | Yassine HAMZAOUI, MALAK AGADI, UIR |
| **Company Supervisor:** | GOURROUM SAMIR, DXC Technology |
| **Field:** | Market Finance — FX Trading Room / Financial IT |
| **Academic Year:** | 2025–2026 |
| **Report Type:** | End-of-Year Internship Report (PFA) |


---

# Abstract

Financial institutions operating in foreign exchange markets require accurate, timely, and auditable calculations of Profit and Loss (P&L) across open and closed positions. This report presents the design and implementation of a data engineering platform for FX P&L monitoring, using the Moroccan Dirham (MAD) as its accounting reference currency.

The proposed architecture separates trade ingestion, SQL transformations, financial calculations, orchestration, data quality controls, and dashboard presentation. The calculation layer addresses Mark-to-Market (MTM) valuation and realized P&L methodologies, including First-In-First-Out (FIFO) and Weighted Average Cost (WAC). The wider platform design includes Apache Airflow orchestration, PostgreSQL persistence, dbt transformations, market-data integration, risk indicators, strategy backtesting, and an operational dashboard.

The report describes the business context, requirements, architecture, data model, implementation approach, testing strategy, and deployment considerations. Numerical examples are used to explain the principal financial calculations. Any implementation results, test outcomes, or integrations should be interpreted in accordance with the evidence available from the project environment.

**Keywords:** FX P&L, Data Engineering, PostgreSQL, dbt, Apache Airflow, FIFO, Weighted Average Cost, Mark-to-Market, Risk Analysis, Backtesting.

# Acknowledgments

I would like to express my sincere gratitude to my academic supervisor at the International University of Rabat for the guidance and support provided throughout this project. I also thank DXC Technology for the professional context that helped shape the project's data engineering focus. Finally, I thank the members of the evaluation committee for their time and consideration.

# Contents

**Abstract** **i**

**Acknowledgments** **ii**

**1 Introduction** **1**
- 1.1 General Context . . . 1
- 1.2 Problem Statement . . . 1
- 1.3 Project Objectives . . . 2
- 1.4 Project Scope . . . 2
- 1.5 Report Structure . . . 2

**2 Business Fundamentals of FX Profit and Loss** **3**
- 2.1 The Foreign Exchange Market . . . 3
- 2.2 Accounting Currency and Market Rates . . . 3
- 2.3 Illustrative Trade Dataset . . . 4
- 2.4 Mark-to-Market Calculation . . . 4
- 2.5 Realized and Unrealized P&L . . . 5
- 2.6 Position Management: FIFO and Weighted Average Cost . . . 5

**3 Requirements Analysis** **7**
- 3.1 Stakeholders . . . 7
- 3.2 Functional Requirements . . . 7
- 3.3 Non-Functional Requirements . . . 8

3.4 Business Rules . . . . . 8

**4 System Architecture** **9**
4.1 Architectural Approach . . . . . 9
4.2 End-to-End Pipeline . . . . . 9
4.3 Technology Stack . . . . . 10
4.4 Deployment Topology . . . . . 10

**5 Data Modeling** **11**
5.1 Entity-Relationship Diagram . . . . . 11
5.2 Data Dictionary . . . . . 11
5.3 Calculation Layer Class Diagram . . . . . 13
5.4 Use Case Diagram . . . . . 13
5.5 Sequence Diagram: End-of-Day Processing . . . . . 14
5.6 FIFO Matching Activity Diagram . . . . . 14

**6 Implementation Approach** **15**
6.1 Database Schema Design . . . . . 15
6.2 Idempotent Ingestion . . . . . 15
6.3 Transformation Layer . . . . . 16
6.4 Calculation Engine . . . . . 16
6.5 Workflow Orchestration . . . . . 17
6.6 Data Quality Controls . . . . . 18

**7 Advanced Analytical Extensions** **19**
7.1 Market Data Integration . . . . . 19
7.2 Exposure and Risk Indicators . . . . . 19
7.3 Market Structure Signals . . . . . 19
7.4 Strategy Backtesting . . . . . 20
7.5 Operational Dashboard . . . . . 20

**8 Validation and Testing** **21**
8.1 Testing Methodology . . . . . 21
8.2 Illustrative Unit Test . . . . . 21
8.3 Reconciliation Controls . . . . . 22
8.4 Test Evidence . . . . . 22

**9 Deployment and Reproducibility** **23**
9.1 Containerized Environment . . . . . 23
9.2 Environment Configuration . . . . . 23
9.3 Version Control . . . . . 23

**10 Project Management** **24**
10.1 Proposed Work Plan . . . . . 24
10.2 Risk Management . . . . . 25

**11 Results and Discussion** **26**
11.1 Expected Deliverables . . . 26
11.2 Engineering Decisions . . . 26
11.3 Limitations . . . 27

**12 Conclusion and Future Work** **28**
12.1 Conclusion . . . 28
12.2 Future Work . . . 28

**Glossary** **29**

# List of Figures

4.1 Proposed end-to-end data engineering architecture . . . 9
4.2 Logical deployment topology . . . 10

5.1 Conceptual entity-relationship model . . . 11
5.2 Conceptual class diagram for the calculation and analytics layers . . . 13
5.3 Conceptual use case diagram . . . 13
5.4 Conceptual sequence for end-of-day processing . . . 14
5.5 Conceptual FIFO lot-matching process . . . 14

6.1 Illustrative orchestration workflow . . . 17

8.1 Conceptual Front Office / Back Office reconciliation process . . . 22

# List of Tables

2.1 Illustrative exchange rates used in the case study . . . 3
2.2 Illustrative FX transactions . . . 4
2.3 Illustrative MTM results for directly quoted pairs . . . 4
2.4 Comparison of realized and unrealized P&L . . . 5
2.5 FIFO and weighted-average cost comparison . . . 6

3.1 Project stakeholders . . . 7
3.2 Non-functional requirements . . . 8

4.1 Technology stack and intended responsibilities . . . . . 10

5.1 Selected data dictionary fields . . . . . 12

10.1 Proposed project work plan . . . . . 24
10.2 Project risk register . . . . . 25

Chapter 1

# Introduction

## 1.1 General Context

A trading room is an operational environment in which financial market transactions are executed, monitored, and recorded. In the foreign exchange (FX) market, transactions involve exchanging one currency for another through pairs such as USD/MAD, EUR/MAD, and EUR/USD.

Profit and Loss (P&L) monitoring is an important activity in this environment. Open positions may be revalued using market closing rates, while closed positions generate realized results based on the applicable cost-basis methodology. These calculations must be traceable, reproducible, and consistent with the valuation conventions adopted by the institution.

Data engineering contributes to this process by creating reliable pipelines that collect transactions, validate input data, apply transformation rules, calculate financial indicators, and expose the results through reporting interfaces.

## 1.2 Problem Statement

Manual calculations, disconnected spreadsheets, and poorly integrated systems can create challenges for FX P&L monitoring. Typical issues include:

- Errors in trade data, exchange-rate selection, or currency conversion;
- Limited visibility into realized and unrealized P&L;
- Difficulties reconciling Front Office and Back Office records;
- Insufficient traceability of calculation methods and data sources;
- Repeated or inconsistent processing when a pipeline is restarted.

The project addresses these challenges through a modular data platform that separates ingestion, transformation, calculation, orchestration, validation, and presentation.

## 1.3 Project Objectives

The main objective is to design and develop a data engineering platform for FX P&L monitoring using MAD as the accounting currency.

The specific objectives are to:

1. Structure and store FX trade and market-rate data;
2. Convert eligible transactions into the accounting currency using explicit conversion rules;
3. Calculate MTM and realized P&L using documented methodologies;
4. Automate processing tasks through workflow orchestration;
5. Introduce data quality checks and reconciliation controls;
6. Present financial indicators through a dashboard;
7. Explore risk analysis and strategy backtesting as additional analytical components.

## 1.4 Project Scope

The report uses a controlled case study to explain the financial logic and proposed processing architecture. Production deployment, access to confidential banking data, and formal approval of accounting policies are outside the scope unless separately evidenced by project artifacts.

## 1.5 Report Structure

Chapter 2 introduces FX P&L concepts. Chapter 3 defines the requirements. Chapter 4 presents the architecture. Chapter 5 describes the data model. Chapter 6 discusses implementation. Chapter 7 presents analytical extensions. Chapter 8 covers validation and testing. Chapter 9 addresses deployment and reproducibility. Chapter 10 discusses project management. Chapter 11 presents results and limitations, and Chapter 12 concludes the report.

Chapter 2

# Business Fundamentals of FX Profit and Loss

## 2.1 The Foreign Exchange Market

A currency pair represents the exchange rate between two currencies. In a pair such as USD/MAD, the quote convention typically expresses the number of Moroccan Dirhams corresponding to one US Dollar. The interpretation of a rate must always be consistent with the pair's base and quote currencies.

Cross-currency transactions, such as EUR/USD, may require an additional conversion when the accounting currency is MAD. The conversion method must respect the relevant quote conventions and the institution's approved valuation policy.

## 2.2 Accounting Currency and Market Rates

For consolidated reporting, the platform uses MAD as the accounting reference currency. A valuation process must identify the applicable market rate, its date, its source, and its quote convention.

The following rates are illustrative values for the case study, not verified live market quotations.

| **Currency Pair** | **Illustrative Rate** |
|---|---|
| USD/MAD | 9.9500 |
| EUR/MAD | 10.8000 |
| EUR/USD | 1.0854 |

**Table 2.1:** Illustrative exchange rates used in the case study

In a production environment, the institution should specify the approved rate source, fixing time, calendar, quote convention, and fallback policy. The source and timestamp should be retained for

auditability.

## 2.3 Illustrative Trade Dataset

The following trades are used to demonstrate the calculation concepts. They do not represent verified transactions from a real trading room.

| ID | Pair | Direction | Nominal | Trade Rate |
|---|---|---|---|---|
| T1 | USD/MAD | Buy USD | 1,000,000 | 9.9200 |
| T2 | USD/MAD | Sell USD | 500,000 | 9.9600 |
| T3 | EUR/MAD | Buy EUR | 800,000 | 10.7700 |
| T4 | EUR/USD | Buy EUR | 600,000 | 1.0820 |

**Table 2.2:** Illustrative FX transactions

## 2.4 Mark-to-Market Calculation

For a simple position denominated in the base currency of a directly quoted pair, a simplified MTM calculation is:

$$\text{Long P\&L} = (\text{Closing Rate} - \text{Trade Rate}) \times \text{Nominal}, \tag{2.1}$$

$$\text{Short P\&L} = (\text{Trade Rate} - \text{Closing Rate}) \times \text{Nominal}. \tag{2.2}$$

These formulas assume that the nominal and rate convention make the resulting amount denominated in the intended quote currency. Real-world valuation may require additional adjustments for settlement conventions, fees, funding, accrued amounts, or different product types.

Applying the simplified formula to the directly quoted pairs:

| Trade | Description | Illustrative P&L (MAD) |
|---|---|---|
| T1 | Long 1,000,000 USD/MAD at 9.9200, closing at 9.9500 | +30,000 |
| T2 | Short 500,000 USD/MAD at 9.9600, closing at 9.9500 | +5,000 |
| T3 | Long 800,000 EUR/MAD at 10.7700, closing at 10.8000 | +24,000 |

**Table 2.3:** Illustrative MTM results for directly quoted pairs

The EUR/USD transaction cannot be valued correctly by applying the preceding formula directly in MAD without explicitly defining the currency exposure and conversion method.

The implementation must distinguish the trade's quote currency from the accounting currency and use a consistent valuation policy.

## 2.5 Realized and Unrealized P&L

Realized P&L generally relates to a position or portion of a position that has been closed under the selected accounting methodology. Unrealized P&L represents the valuation change associated with positions that remain open.

| Criterion | Realized P&L | Unrealized P&L |
|---|---|---|
| Basis | Closed quantity and allocated cost basis | Open quantity and current valuation |
| Market sensitivity | Depends on the recognized closing transaction and method | Changes with the valuation rate |
| Calculation inputs | Trade history, matched lots, execution rates | Open position, cost basis, closing rate |
| Reporting | Subject to the institution's accounting policy | Subject to the institution's revaluation policy |

**Table 2.4:** Comparison of realized and unrealized P&L

**Business Rule**

For a reporting definition that uses these two components, total P&L is expressed as:

$$\text{Total P\&L} = \text{Realized P\&L} + \text{Unrealized P\&L}.$$

The system must define the scope and valuation basis of each component to avoid double counting.

## 2.6 Position Management: FIFO and Weighted Average Cost

When multiple purchases occur at different prices before a sale, the selected cost-basis methodology determines how the sold quantity is matched to previous purchases.

| **Criterion** | **FIFO** | **WAC / CMP** |
|---|---|---|
| Matching principle | Oldest eligible lot is consumed first | Eligible inventory is valued using an average cost |
| State maintained | Individual lots and residual quantities | Aggregate quantity and average cost |
| Implementation | Sequential lot matching | Weighted-average recalculation |
| Audit requirements | Requires lot-level traceability | Requires traceability of average-cost updates |

**Table 2.5:** FIFO and weighted-average cost comparison

Both methods require consistent trade ordering, direction handling, precision rules, and explicit behavior for insufficient inventory or short positions. The correct method depends on the applicable business and accounting policy.

Chapter 3

# Requirements Analysis

## 3.1 Stakeholders

| Stakeholder | Primary Interest |
|---|---|
| Trader | Record transactions and monitor position P&L |
| Risk Manager | Review exposure, thresholds, and risk indicators |
| Back Office | Compare transaction records and investigate discrepancies |
| Data Engineering Team | Maintain reliable ingestion, transformations, and orchestration |
| Academic Evaluator | Assess technical implementation and documentation quality |

**Table 3.1:** Project stakeholders

## 3.2 Functional Requirements

**FR1** Record FX trades with pair, direction, nominal, rate, timestamp, and book.

**FR2** Validate and normalize currency pairs and market rates.

**FR3** Calculate MTM P&L for supported instruments and quote conventions.

**FR4** Calculate realized P&L using a configured cost-basis method.

**FR5** Report realized, unrealized, and total P&L without double counting.

**FR6** Orchestrate the daily processing workflow.

**FR7** Detect data quality failures before publishing results.

**FR8** Display selected risk indicators and threshold alerts.

**FR9** Provide an operational dashboard for reviewing calculated results.

**FR10** Support reconciliation of corresponding Front Office and Back Office records where suitable data is available.

## 3.3 Non-Functional Requirements

| Category | Requirement |
|---|---|
| Reliability | Reprocessing should not create duplicate transactions. |
| Financial precision | Monetary calculations should use fixed-point or decimal arithmetic. |
| Traceability | Inputs, rates, calculation dates, and methods should be auditable. |
| Separation of concerns | Orchestration metadata should be separated from business data. |
| Portability | The environment should be reproducible through documented configuration. |
| Testability | Calculation rules should be tested independently using known examples. |
| Security | Credentials and API keys should be stored outside source code. |

**Table 3.2:** Non-functional requirements

## 3.4 Business Rules

**BR1** Every supported valuation must identify its accounting currency and conversion convention.

**BR2** Unrealized P&L must be recalculated when the applicable valuation inputs change.

**BR3** Reported P&L components must have clearly defined scopes and must not be double counted.

**BR4** The cost-basis method must be configured and recorded for reproducibility.

**BR5** Invalid trades, unsupported currency pairs, and missing rates must be flagged rather than silently assigned an arbitrary conversion.

**BR6** Risk alerts must be traceable to the threshold and inputs used to generate them.

Chapter 4

# System Architecture

## 4.1 Architectural Approach

The proposed platform separates ingestion, persistence, transformation, calculation, orchestration, validation, and presentation. This approach allows each component to be tested independently and reduces the risk of embedding financial calculations inside dashboard code.

## 4.2 End-to-End Pipeline

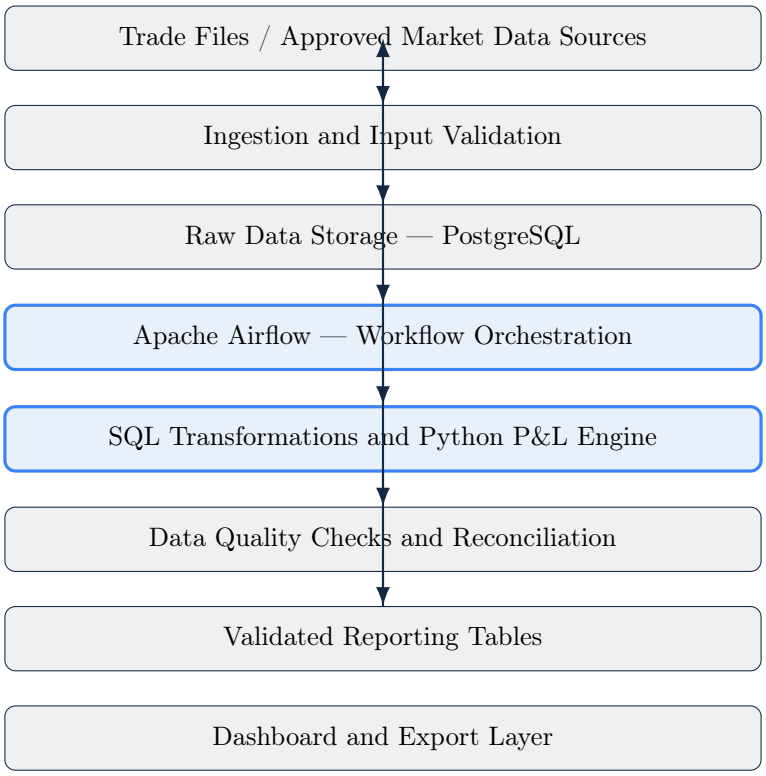



**Figure 4.1:** Proposed end-to-end data engineering architecture

## 4.3 Technology Stack

| Layer | Technology / Approach |
|---|---|
| Ingestion | Python and file/API connectors |
| Persistence | PostgreSQL |
| Transformation | SQL and dbt, where configured |
| Orchestration | Apache Airflow, where configured |
| Calculation engine | Python with explicit decimal precision |
| Dashboard | Streamlit and Plotly, where implemented |
| Containerization | Docker Compose, where configured |
| Version control | Git and GitHub |

**Table 4.1:** Technology stack and intended responsibilities

## 4.4 Deployment Topology

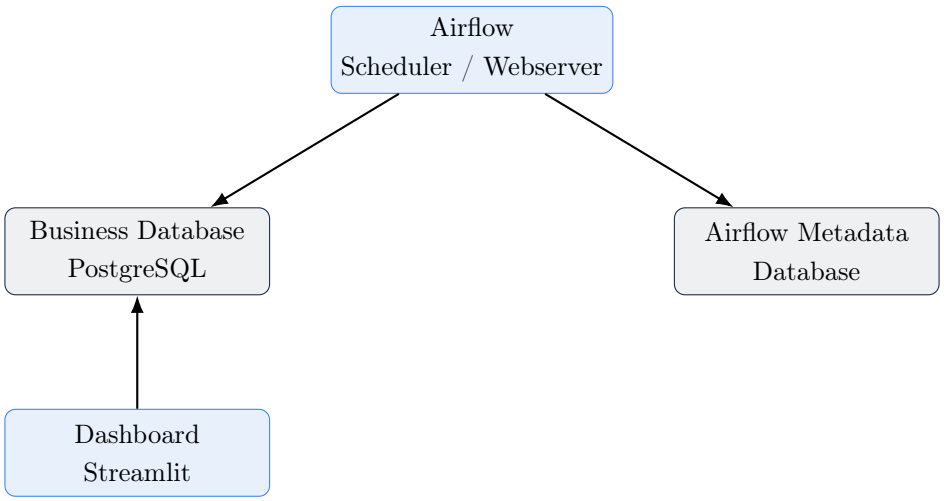



**Figure 4.2:** Logical deployment topology

This diagram illustrates a possible deployment design. The actual number of containers, ports, credentials, and dependencies must match the project's Docker Compose configuration.

Chapter 5

# Data Modeling

## 5.1 Entity-Relationship Diagram

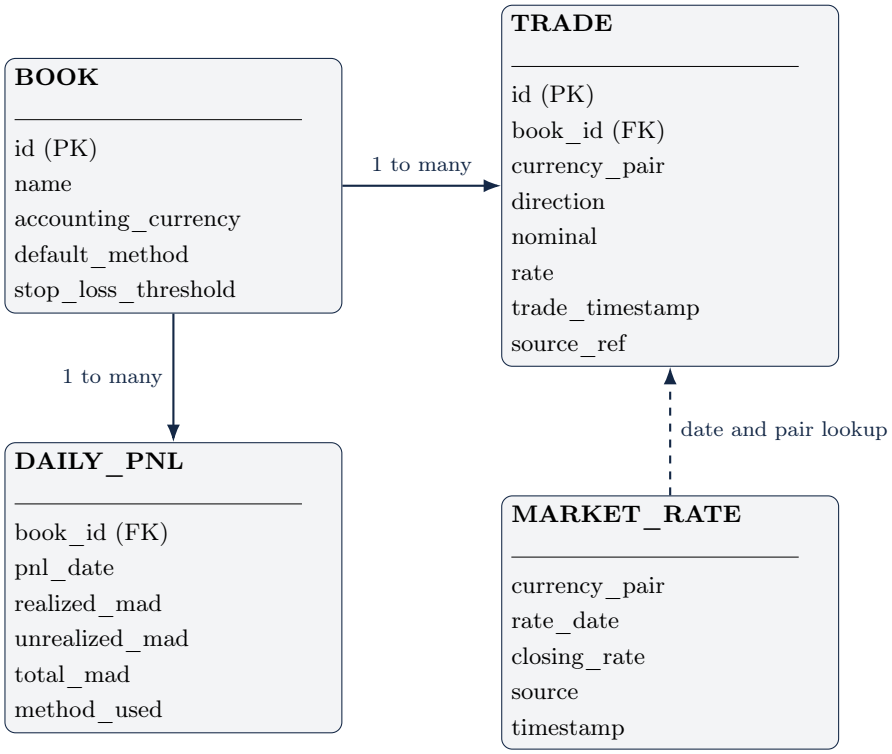



**Figure 5.1:** Conceptual entity-relationship model

The diagram is conceptual. The final database schema should define primary keys, foreign keys, uniqueness constraints, numeric precision, and nullability explicitly. The relationship between trades and market rates may be implemented through a lookup rather than a direct foreign key, depending on the valuation model.

## 5.2 Data Dictionary

| Table | Column | Description |
|---|---|---|
| book | id | Primary key for a trading book. |
| book | name | Book identifier or display name. |
| book | accounting_currency | Currency used for reporting. |
| book | default_method | Configured realized P&L cost-basis method. |
| trade | source_ref | External reference used to identify duplicate records. |
| trade | currency_pair | Currency pair associated with the transaction. |
| trade | direction | Transaction direction under a defined convention. |
| trade | nominal | Transaction quantity in the defined base currency. |
| trade | rate | Execution rate under the pair's quote convention. |
| market_rate | closing_rate | Rate selected for the relevant valuation date. |
| market_rate | source | Origin of the market-rate record. |
| daily_pnl | realized_mad | Realized result in the accounting currency. |
| daily_pnl | unrealized_mad | Unrealized result in the accounting currency. |
| daily_pnl | total_mad | Total result under the reporting definition. |

**Table 5.1:** Selected data dictionary fields

## 5.3 Calculation Layer Class Diagram

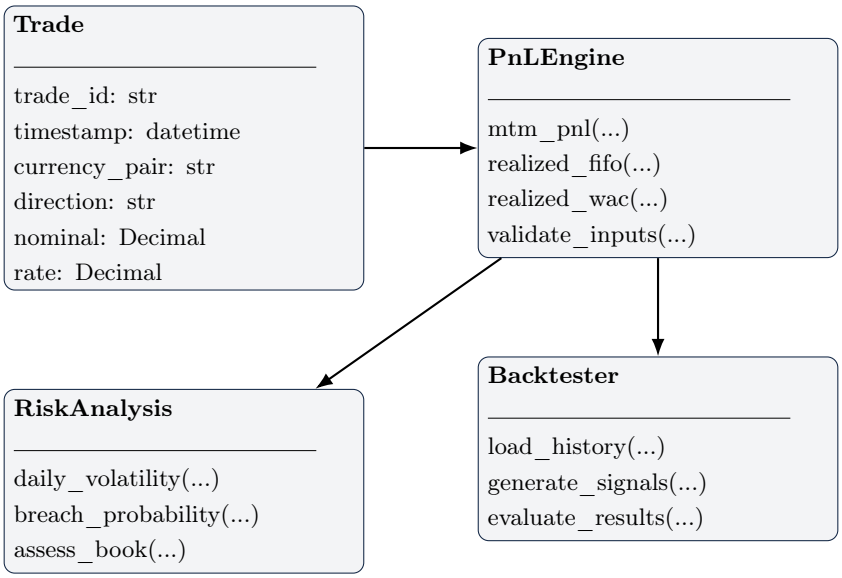



**Figure 5.2:** Conceptual class diagram for the calculation and analytics layers

## 5.4 Use Case Diagram

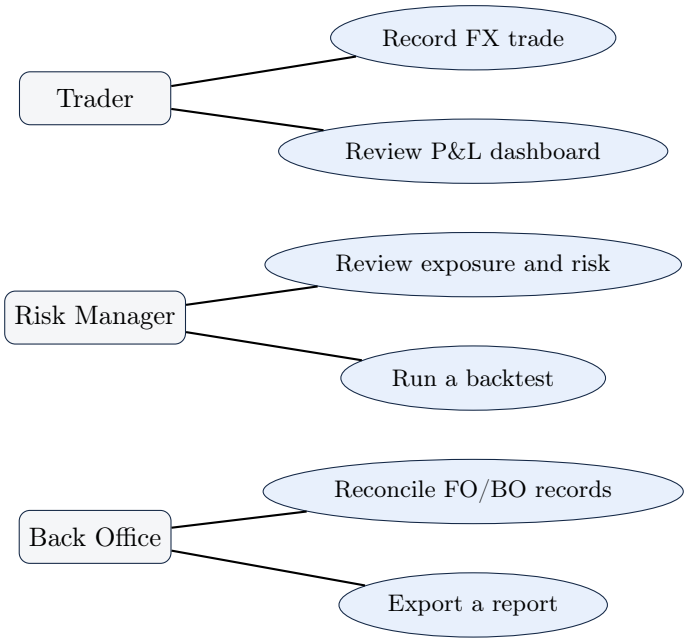



**Figure 5.3:** Conceptual use case diagram

## 5.5 Sequence Diagram: End-of-Day Processing

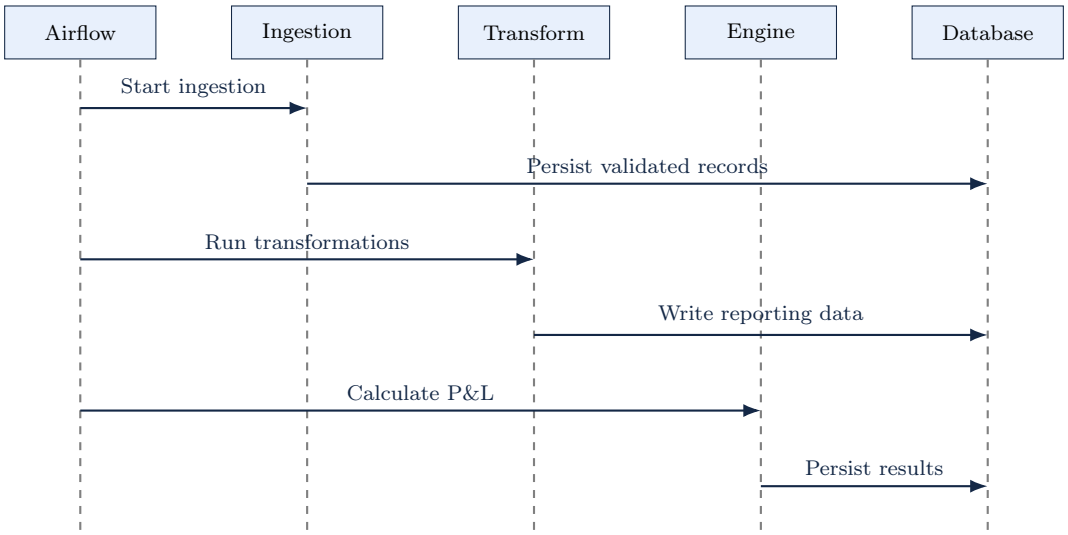



**Figure 5.4:** Conceptual sequence for end-of-day processing

## 5.6 FIFO Matching Activity Diagram

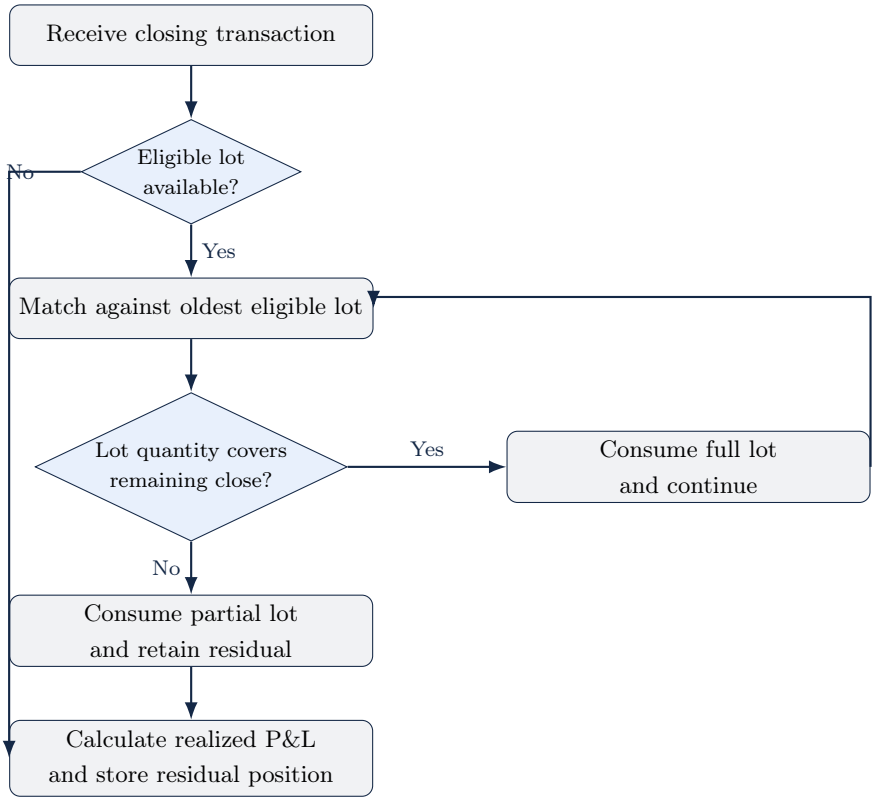



**Figure 5.5:** Conceptual FIFO lot-matching process

Chapter 6

# Implementation Approach

## 6.1 Database Schema Design

The database should distinguish immutable transaction records from derived calculation results. Monetary values and rates should use suitable fixed-point numeric types. Constraints should be used to reject invalid states where the business rule can be expressed safely at the database level.

```
CREATE TABLE daily_pnl (
    book_id        INTEGER NOT NULL REFERENCES book(id),
    pnl_date       DATE NOT NULL,
    realized_mad   NUMERIC(18,2) NOT NULL,
    unrealized_mad NUMERIC(18,2) NOT NULL,
    total_mad      NUMERIC(18,2) NOT NULL,
    method_used    VARCHAR(4) NOT NULL
        CHECK (method_used IN ('FIFO', 'WAC')),
    PRIMARY KEY (book_id, pnl_date),
    CHECK (
        total_mad = realized_mad + unrealized_mad
    )
);
```

**Listing 6.1:** Illustrative daily PL table

This example illustrates a database-level consistency constraint. The actual implementation should confirm the chosen rounding policy and ensure that the same precision rules are applied in the application and database layers.

## 6.2 Idempotent Ingestion

An ingestion process should identify records using a stable source reference or a suitable composite key. Database uniqueness constraints and conflict-handling logic can prevent duplicate records when a file is reprocessed or a workflow task is retried.

A robust ingestion workflow should also record rejected rows, validation errors, input file identifiers, ingestion timestamps, and processing status.

## 6.3 Transformation Layer

A transformation layer can standardize column names, normalize currency-pair notation, validate directions, and join trades to the correct market-rate records. Cross-currency conversion should be implemented explicitly rather than assuming that every input rate can be used directly as a MAD valuation rate.

```sql
SELECT
    currency_pair,
    rate_date,
    closing_rate,
    source
FROM market_rate
WHERE closing_rate <= 0
   OR closing_rate IS NULL;
```

**Listing 6.2:** Illustrative market-rate validation query

A production-quality transformation should additionally detect missing rates, duplicate rates for the same pair and date, stale rates, and unsupported currency pairs.

## 6.4 Calculation Engine

FIFO and WAC should be implemented as independent functions with explicit input contracts. Financial quantities should use decimal arithmetic, and the engine should validate transaction ordering, trade direction, available inventory, and partial fills.

```python
from collections import deque
from decimal import Decimal

def realized_fifo(buy_lots, sell_quantity, sell_rate):
    remaining = Decimal(str(sell_quantity))
    sell_rate = Decimal(str(sell_rate))
    queue = deque(sorted(
        buy_lots,
        key=lambda lot: lot["timestamp"]
    ))

    realized = Decimal("0")
    residual = []

    while queue and remaining > 0:
        lot = queue.popleft()
        quantity = Decimal(str(lot["quantity"]))
        buy_rate = Decimal(str(lot["rate"]))

        matched = min(quantity, remaining)
        realized += (sell_rate - buy_rate) * matched
        remaining -= matched
        quantity -= matched
```

```
24
25          if quantity > 0:
26              residual.append({
27                  **lot,
28                  "quantity": quantity
29              })
30
31      residual.extend(queue)
32
33      if remaining > 0:
34          raise ValueError(
35              "Insufficient eligible inventory for FIFO matching."
36          )
37
38      return realized, residual
```

**Listing 6.3:** Illustrative FIFO matching logic

This is an illustrative long-position matching example. A complete implementation must also define short-position handling, currency conversion, precision and rounding, fees, and the treatment of unmatched transactions.

## 6.5 Workflow Orchestration

The intended end-of-day workflow can be represented as:

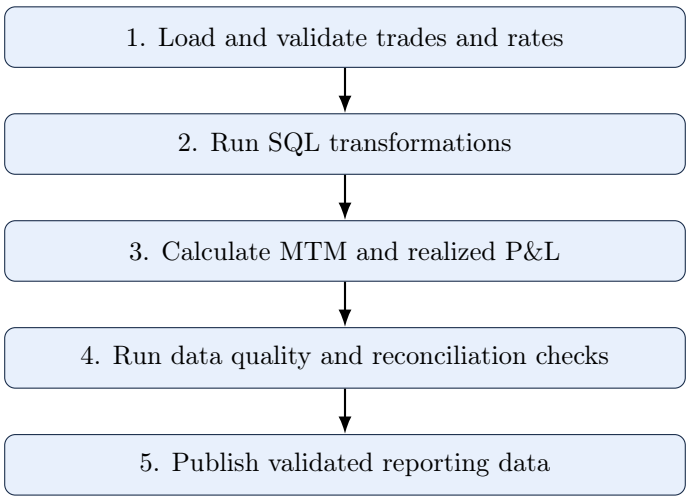



**Figure 6.1:** Illustrative orchestration workflow

Apache Airflow can express the dependencies between these tasks, record execution history, and support retries. Task retries should be safe only when the underlying operations are designed to be idempotent.

## 6.6 Data Quality Controls

Data quality checks should cover:

- Unique trade references and valid transaction timestamps;
- Positive nominal amounts and valid exchange rates;
- Supported currency pairs and directions;
- Required market rates for the valuation date;
- Consistency between realized, unrealized, and total P&L;
- Reconciliation differences between available source systems.

A failed validation should prevent affected results from being published as validated output. The pipeline should preserve enough diagnostic information to identify and correct the underlying issue.

Chapter 7

# Advanced Analytical Extensions

## 7.1 Market Data Integration

A market-data connector can retrieve current or historical rates from an approved provider. The connector should document API limits, timestamps, rate conventions, missing-data handling, and the distinction between demonstration data and rates approved for accounting valuation.

A market-data API response should not automatically be treated as an official accounting fixing without explicit approval of the source and valuation policy.

## 7.2 Exposure and Risk Indicators

A risk component can summarize the open exposure of a book and compare selected metrics with configured thresholds. A simple statistical indicator may estimate the probability of crossing a threshold under a stated distributional assumption.

For example, if a normally distributed P&L change is assumed with mean $\mu$ and standard deviation $\sigma$, the probability of ending below a threshold $L$ can be expressed as:

$$P(X < L) = \Phi\left(\frac{L - \mu}{\sigma}\right),$$

where $\Phi$ is the standard normal cumulative distribution function.

This is a simplified model, not a guarantee of future outcomes. Its usefulness depends on the quality of historical observations, the validity of the distributional assumption, and the chosen time horizon. A production risk process would require calibration, backtesting, stress scenarios, and governance.

## 7.3 Market Structure Signals

A technical-analysis module may identify swing highs, swing lows, and break-of-structure events from historical candles. Such rules can be used to generate descriptive signals for research or

demonstration.

```
if last_close > previous_swing_high:
    signal = "bullish_break_of_structure"
elif last_close < previous_swing_low:
    signal = "bearish_break_of_structure"
else:
    signal = "no_confirmed_break"
```

**Listing 7.1:** Illustrative market-structure classification

The signal should be treated as a rule-based indicator rather than a validated prediction of future prices.

## 7.4 Strategy Backtesting

A backtesting component can replay historical market data chronologically, generate signals, simulate entries and exits, and calculate results after defined transaction costs. To avoid look-ahead bias, each decision must use only information available at the time of the simulated trade.

A credible backtest should document the data source, time zone, candle construction, spread and commission assumptions, slippage, position sizing, risk limits, and out-of-sample evaluation.

## 7.5 Operational Dashboard

A dashboard can present P&L summaries, open exposure, market rates, data quality status, and reconciliation differences. It should read validated outputs from the data layer rather than independently reimplementing the financial calculation logic.

The interface should distinguish clearly between illustrative values, simulated values, live market observations, and results approved for reporting.

Chapter 8

# Validation and Testing

## 8.1 Testing Methodology

Testing should be organized around independently defined examples and business rules. Unit tests can verify individual calculation functions, integration tests can validate database interactions, and end-to-end tests can verify the processing workflow.

The same calculation should not be considered independently validated merely because two components agree if both use the same incorrect assumptions. Reference examples should therefore be recalculated manually or checked against an independently specified expected result.

## 8.2 Illustrative Unit Test

```python
from decimal import Decimal

def mtm_long(nominal, trade_rate, closing_rate):
    nominal = Decimal(str(nominal))
    trade_rate = Decimal(str(trade_rate))
    closing_rate = Decimal(str(closing_rate))
    return (closing_rate - trade_rate) * nominal

def test_mtm_long_usd_mad():
    result = mtm_long(
        "1000000",
        "9.9200",
        "9.9500"
    )
    assert result == Decimal("30000.0000")
```

**Listing 8.1:** Example unit test for a directly quoted pair

This example validates one simplified calculation. The complete test suite should include short positions, partial closes, insufficient inventory, missing rates, unsupported pairs, rounding boundaries, and repeated ingestion.

## 8.3 Reconciliation Controls

Reconciliation should compare records using stable identifiers and explicit matching rules. A reconciliation result should identify matched records, missing records, duplicate records, and differences in quantity, rate, date, or calculated P&L.

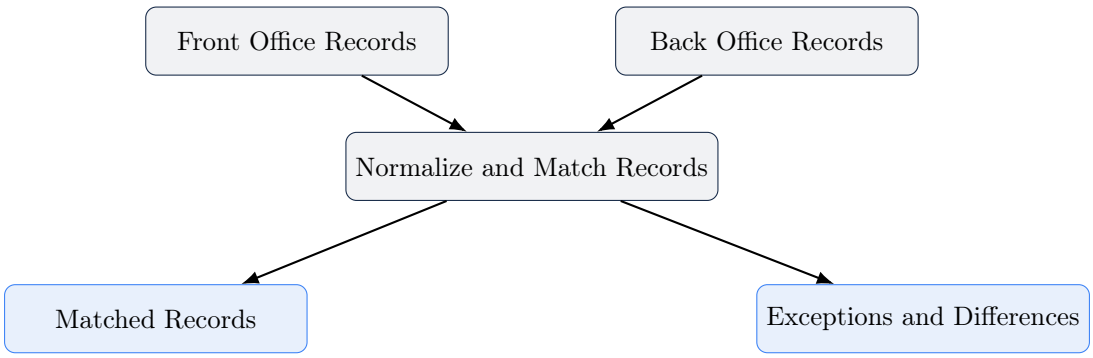



**Figure 8.1:** Conceptual Front Office / Back Office reconciliation process

## 8.4 Test Evidence

The final report should include only test outcomes that can be reproduced from the actual project environment. For each test suite, record the command, environment, date, number of tests executed, number passed, number failed, and any known limitations.

**Important Note**

Do not report a test count, runtime, production integration, or successful validation unless it is supported by recorded output or other reproducible project evidence.

Chapter 9

# Deployment and Reproducibility

## 9.1 Containerized Environment

Docker Compose can define the services required by the application, including the business database, orchestration services, and dashboard. The precise service list must follow the actual project configuration.

An example startup command is:

```
docker compose up -d
docker compose ps
docker compose logs --tail=100
```

**Listing 9.1:** Example Docker Compose startup

The ingestion, transformation, calculation, and dashboard commands should be documented according to the actual repository structure. Commands should not be copied into the final report until they have been tested in the target environment.

## 9.2 Environment Configuration

Database credentials, API keys, and other secrets should be supplied through environment variables or a suitable secrets manager. Local environment files should be excluded from version control, and the report should not expose real credentials.

## 9.3 Version Control

Git provides a history of code changes and supports collaboration and reproducibility. The repository should include installation instructions, dependency versions, configuration examples without secrets, and documented steps for initializing the database and running tests.

Chapter 10

# Project Management

## 10.1 Proposed Work Plan

The following table presents a suggested project structure. Actual durations and completion dates should be adjusted to match the documented project timeline.

| Phase | Activity | Duration | Deliverable |
|---|---|---|---|
| P1 | Business analysis | 3 weeks | Scope and business rules |
| P2 | Data modeling and architecture | 3 weeks | Data model and diagrams |
| P3 | Pipeline development | 5 weeks | Ingestion and calculation components |
| P4 | Orchestration and quality | 3 weeks | Workflow and validation checks |
| P5 | Analytical extensions | 3 weeks | Risk and backtesting components |
| P6 | Integration and documentation | 2 weeks | Integrated application and report |

**Table 10.1:** Proposed project work plan

## 10.2 Risk Management

| Risk | Potential Impact | Mitigation |
|---|---|---|
| Incorrect currency conversion | Incorrect P&L valuation | Explicit quote conventions and test cases |
| Duplicate ingestion | Inflated transaction totals | Stable source keys and uniqueness constraints |
| Pipeline failure | Incomplete reporting | Monitoring, retries, and failure alerts |
| Rounding inconsistencies | Differences between components | Centralized precision and rounding rules |
| Look-ahead bias | Misleading backtest results | Chronological replay and data availability checks |
| Credential exposure | Unauthorized access | Environment configuration and secret management |

**Table 10.2:** Project risk register

Chapter 11

# Results and Discussion

## 11.1 Expected Deliverables

The project is structured around the following deliverables:

- A defined data model for trades, market rates, and P&L results;
- An ingestion workflow with validation and duplicate protection;
- A calculation layer for supported MTM and realized P&L methods;
- An orchestrated processing workflow;
- Data quality and reconciliation checks;
- A dashboard for reviewing validated output;
- Optional risk and backtesting components, subject to the evidence available in the implementation.

## 11.2 Engineering Decisions

Several design choices are important for a financial data application. Decimal arithmetic reduces binary floating-point representation problems. Separating orchestration metadata from business records improves operational clarity. Independent calculation functions make financial logic easier to test. Explicit data contracts help prevent unsupported values from flowing into reporting tables.

These choices improve the system's design, but they do not by themselves prove production readiness. Production readiness also requires operational monitoring, security review, performance testing, access controls, documented recovery procedures, and validation against the institution's approved business rules.

## 11.3 Limitations

The case study uses illustrative rates and trades. It does not establish that the platform has been approved for real financial reporting or connected to production banking systems. Market-data licensing, accounting policy, multi-currency product support, historical data quality, and operational governance require additional validation.

Any strategy signal or backtest should be treated as an analytical experiment rather than a guarantee of future performance or a trading recommendation.

Chapter 12

# Conclusion and Future Work

## 12.1 Conclusion

This report presents a data engineering approach to FX P&L monitoring, combining structured ingestion, persistent storage, financial calculation logic, workflow orchestration, data quality controls, and dashboard reporting.

The architecture emphasizes reproducibility, precision, traceability, and separation of concerns. It also identifies the importance of explicit currency-conversion rules, well-defined cost-basis methods, and independently testable financial calculations.

The project provides a foundation for further development, while production use would require validation against approved accounting policies, reliable market data, operational controls, and the actual implementation evidence.

## 12.2 Future Work

Potential extensions include:

- Full multi-currency and multi-book support;
- Additional financial products and settlement conventions;
- Automated reconciliation reports in PDF or spreadsheet format;
- Stronger monitoring, alerting, and recovery mechanisms;
- Stress testing and more comprehensive risk measures;
- Out-of-sample backtesting and explicit transaction-cost modeling;
- Role-based access control and audit-log integration.

# Glossary

| | |
|---|---|
| **BOS** | Break of Structure, a price movement beyond a previously identified swing level. |
| **DAG** | Directed Acyclic Graph, a representation of dependent tasks in a workflow. |
| **dbt** | Data Build Tool, used to manage SQL-based data transformations. |
| **FO** | Front Office, the part of a financial institution associated with trading and client-facing activity. |
| **FIFO** | First-In-First-Out, a cost-basis method that consumes the oldest eligible lot first. |
| **MTM** | Mark-to-Market, the valuation of a position using a selected market price or rate. |
| **P&L** | Profit and Loss. |
| **WAC/CMP** | Weighted Average Cost / Coût Moyen Pondéré. |

# Bibliography


[1] PostgreSQL Global Development Group, *PostgreSQL Documentation*. https://www.postgresql.org/docs/

[2] Apache Software Foundation, *Apache Airflow Documentation*. https://airflow.apache.org/docs/

[3] dbt Labs, *dbt Documentation*. https://docs.getdbt.com/

[4] Python Software Foundation, *Python Documentation*. https://docs.python.org/3/

[5] Twelve Data, *API Documentation*. https://twelvedata.com/docs